Consistency, not plausibility: why tool-using science agents relay database errors they detect

[Authors]

Abstract

AI agents that query scientific databases are increasingly trusted to retrieve and reason over records, yet databases contain errors. We asked when tool-using agents relay a wrong database value into their answer. Across a live deployment of the Biomni agent, a controlled benchmark of ten models from five developers, and 79 protein sequence errors that UniProt itself later corrected, agents routinely detected errors and relayed them anyway. Whether an agent overrode a wrong value depended less on its plausibility than on whether the record agreed with itself: adding one field consistent with the error raised relay from 63% to 91% for one model, and historically corrected errors, whose records were internally consistent, were relayed in 90–100% of runs regardless of size. Agents that computed the correct value from the record's own formula still answered from the record. An independent second source removed relay entirely when applied silently (0 of 702 runs), but showing agents the conflict let them restore the original value.

Language-model agents are moving from answering questions to doing research. Systems such as ChemCrow [1] and Biomni [2] plan multistep analyses, query scientific databases and compute on what they retrieve. Their answers can only be as good as the records they read, and those records contain errors. Misannotation is widespread in several major protein databases [3], and even curated resources revise their entries: UniProt archives every earlier version of every entry [4], and of the 300 reviewed human proteins in our benchmark, 167 have at least one superseded sequence. An agent that relays a database value inherits its error unless it notices the error and acts on it.

How language models handle wrong information in their context has been studied mainly as a conflict between the context and the model's own knowledge [5, 6]. Models often adopt incorrect retrieved content, less often the further it departs from the truth [7], and they accept counter-evidence that is coherent and convincing even when it contradicts their memory [8]. Concurrent work shows that tool-using agents adopt corrupted tool returns and may question a return, or even state the correct value, while passing the corrupted content on [9]. These studies corrupt passages of text or single returned values. Scientific database records are different. A value sits among related fields that can confirm or contradict it, and the record often contains what is needed to derive the value independently: a protein's sequence fixes its length and mass, and a compound's formula fixes its weight. Whether agents use this structure, whether errors that occurred in real databases behave like constructed ones, and which corrections actually change what an agent reports, are open questions.

We studied them in five steps. We ran a deployed biomedical agent against live database APIs and altered the responses in transit. We built a controlled benchmark of protein and small-molecule questions and ran ten models from five developers. We manipulated the internal consistency of the records and the derivation routes they offer. We presented errors that UniProt itself later corrected. And we tested corrections: checks of each record against its own derivations, an independent second source, and three ways of presenting a correction to the agent. Where we made predictions, we registered them before running the experiments, and we report the ones that failed.

Across these experiments, agents noticed wrong values far more often than they acted on them. Whether a wrong value was relayed depended less on how plausible it was than on whether the rest of the record agreed with it: adding one agreeing field raised relay, and making the record disagree with itself lowered it, with the wrong value unchanged. A derivation route let agents detect an error, and often compute the correct value, without changing what they answered. Errors that UniProt later corrected, each internally consistent, were relayed in 90–100% of runs, whatever the size of the revision. An independent second source removed most of these errors; correcting the record silently removed all of them in our tests, whereas showing the agent the original value invited it back.

Results

A deployed agent relays corrupted live database values

To test whether the problem arises outside a benchmark, we ran Biomni [2], an open-source biomedical agent, against the live UniProt and PubChem APIs and altered one field of selected responses in transit with an intercepting proxy (Methods). Biomni retrieved and processed records with its own code and tools; we changed only what the databases returned. Each of 300 tasks ran once without corruption, once with an impossible value and once with a plausible value within 25% of the true one, for two models, with hypotheses registered in advance (OSF vq5cp).

The corrupted value reached the agent in 95–96% of corrupted runs; in the remainder the agent answered through another route and never requested the corrupted field. Impossible values were almost always caught: they were relayed in 5 of 287 runs with DeepSeek V4.1 Flash and 6 of 286 with Gemini 3.8 Flash. Plausible values were relayed in 28% (80/285) and 24% (69/288) of runs, far fewer than in the controlled benchmark below (McNemar $p < 10^{-30}$ for both models) but not rare.

Where plausible values were relayed contradicted our second prediction. We had expected fields with an exact derivation route to be protected; instead they were relayed more often (odds ratio 5.6 and 3.6). Protein length and mass, which follow exactly from the sequence, were relayed in 46–68% of plausible runs, organism in 0–14% and names in 0–8%, and numeric chemistry fields almost never, because Biomni's own workflow recomputes them with RDKit. A derivation route protected a value only when the agent's workflow actually passed through it. A third prediction, about which concern an agent raises first, was supported for one model and not the other (Supplementary Information).

Agents notice errors and relay them anyway

To separate what agents detect from what they report, we built a controlled benchmark of 300 human proteins from UniProt and 300 small molecules from PubChem, with lookup and threshold questions across ten templates, and ran ten models from five developers (Methods). Agents retrieved records through a lookup tool. In corrupted conditions one field of the target record held either an impossible value, such as a mass of 45 Da for a 228-residue protein, or a plausible one, scaled down or up by about a fifth.

Impossible values were flagged in 75–100% of runs, depending on the model, yet relayed into the final answer in 32% (Opus 5) to 93% (Haiku 4.5) of protein runs and 38% (DeepSeek V4.1 Flash) to 91% (Haiku 4.5) of chemistry runs (Fig. 2). Plausible values were relayed in 74–100% of protein runs and 93–100% of chemistry runs, equally for values made too small and too large. These plausible numeric corruptions scaled paired fields together, so the record stayed internally consistent, whereas

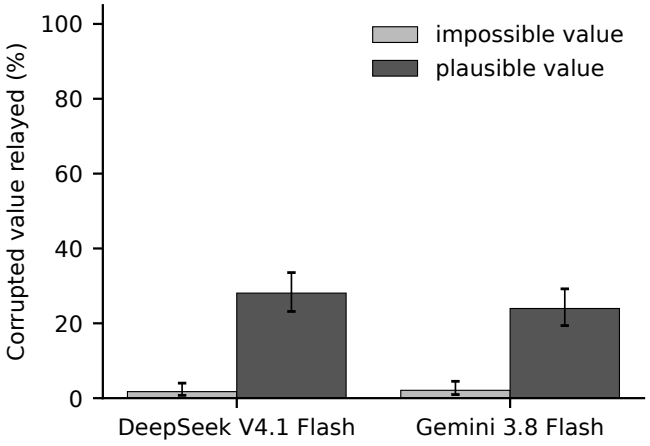


Figure 1: **A deployed agent relays plausible corrupted values.** Share of delivered corrupted runs in which Biomni relayed the corrupted value, by model and kind of corruption (95% Wilson intervals).

impossible values changed one field and contradicted its partner; we return to this distinction below. Models ranked similarly in both domains (Spearman $\rho = 0.71$, $p = 0.02$). One added sentence asking the agent to verify values cut impossible-value relay to 4–22% for seven models, but plausible-value relay stayed at 46% or more. When the tool returned the full record, including the sequence or formula, agents corrected values that the record contradicted: organism errors fell from 100% to 0% for all three models tested, and Opus 5's relay of impossible and plausible values fell from 33% to 1% and from 75% to 9%.

What do agents cite when they flag a value? We classified every concern with a model-based classifier validated against two human annotators (inter-annotator Cohen's κ 0.73–0.89; classifier κ 0.65–0.87 on a held-out consensus set; Methods). Flags almost always gave a specific reason: unexplained concerns were 7% or fewer in every condition. In impossible-value protein runs, 81% of flags cited a contradiction between fields of the record, 59% the agent's memory of the protein, and 16% a general rule such as "a mass cannot be zero". Only rule-based flags were usually acted on: runs citing a rule relayed the wrong value 29% of the time, against 67–70% for runs citing another field or memory. Chemistry showed no such advantage (59% against 48–60%). Agents noticed; noticing was not enough.

Self-consistent errors pass, even when impossible

The full-record result suggests that agents correct a value when the record contradicts it. To measure how large a contradiction must be, we scaled one field of the full record by factors from $\times 0.005$ to $\times 200$ and scaled its paired field alike, so that only the sequence or formula disagreed with the value (Methods; 40 tasks per factor and direction). Every model overrode large contradictions, but the change at which half of runs overrode differed by model: within about 20% for Opus 5, Gemini 4 Argon and Gemini 3.8 Flash, 50–80% for DeepSeek V4.1 Flash, and roughly twofold for Sonnet 5 ($\times 0.54$ and $\times 2.30$ for proteins; Fig. 3). Within each model, boundaries were similar for proteins and chemistry. They did not transfer to new compounds, however: in a registered follow-up, the same corruption applied to 60 other compounds was relayed more often than the curves predicted for three of five models (5 of 10 prediction intervals met).

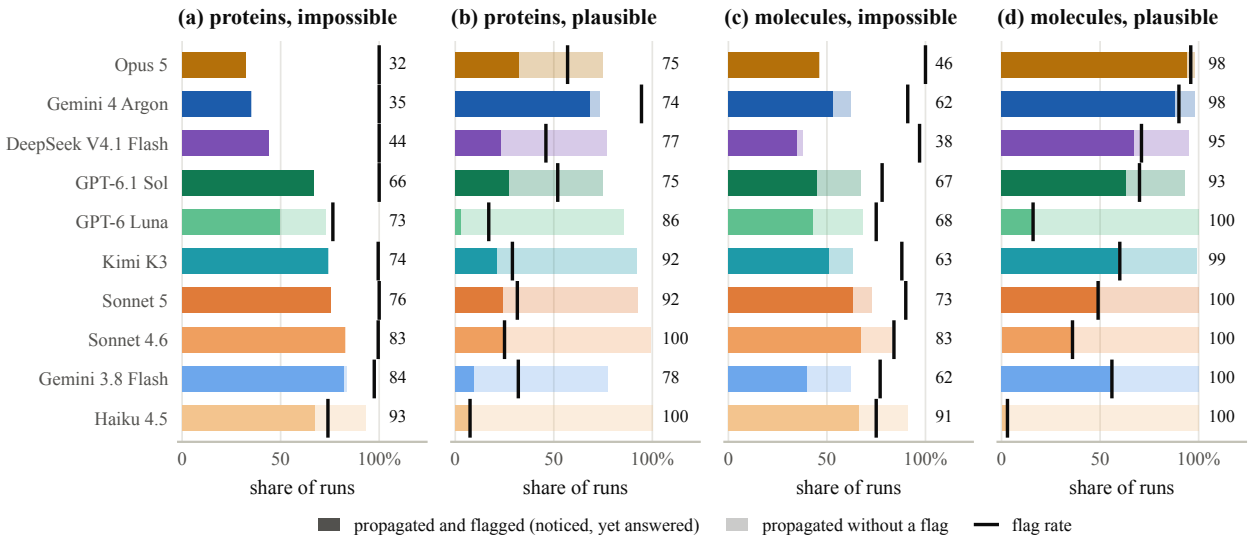


Figure 2: **Noticed yet answered.** Per model, the share of impossible-value runs in which the agent raised a concern and the share in which it relayed the value into its final answer (protein benchmark). [Check the caption against the figure panels.]

What if the record contains no contradiction at all? We edited the sequence, or substituted the structure, and recomputed every derived field, so that the record was internally consistent and passed every derivation check (Methods). With the full record, Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash relayed these coherent plausible errors in 94–98% of runs that shortened a protein, 70–96% of runs that lengthened one and 93–98% of chemistry runs, compared with 9% for Opus 5 when the full record contradicted the value. Consistency carried impossible values too. A three-residue sequence presented as a known human protein, with matching length and mass, was relayed in 94–100% of runs from the summary record, against 32–44% for the benchmark's impossible values, which contradicted their paired field; the two kinds of value differ, so the comparison is indicative. With the full sequence in view, relay of the three-residue protein fell to 12–42%, while a methane structure substituted for a drug was relayed in 45–100% of runs.

Agents did use sequence-level evidence when it was visible. When we lengthened sequences by repeating their last 60 residues, Opus 5 and Argon relayed the wrong length in 40% and 44% of runs, against 76% and 70% when the added residues were a natural segment of another protein. DeepSeek mentioned the repeat in 48 of 50 runs and relayed the wrong length in 92% of them.

A derivation route enables detection; agreement decides override

Why did some contradictions lead to correction and others not? We registered a test with three chemistry fields not used before, chosen by how exactly they follow from the formula and structure in the record: monoisotopic mass (exact), topological polar surface area (approximate) and PubChem complexity (no route; Methods). We predicted that relay of plausible errors would rise from the exact to the routeless field, and that relay of monoisotopic-mass errors would match each model's molecular-weight boundary. The ordering was present but negligible (96%, 99.6% and 100% across four models; route-rank one-sided $p = 0.007$), and the quantitative prediction failed: four models relayed monoisotopic-mass errors in 90–100% of runs where 27–59% was predicted.

The route did change detection. With the full record, agents raised a concern about a corrupted monoisotopic mass in 36–60 of 60 runs per model, against 0–6 of 60 for uncorrupted records; TPSA

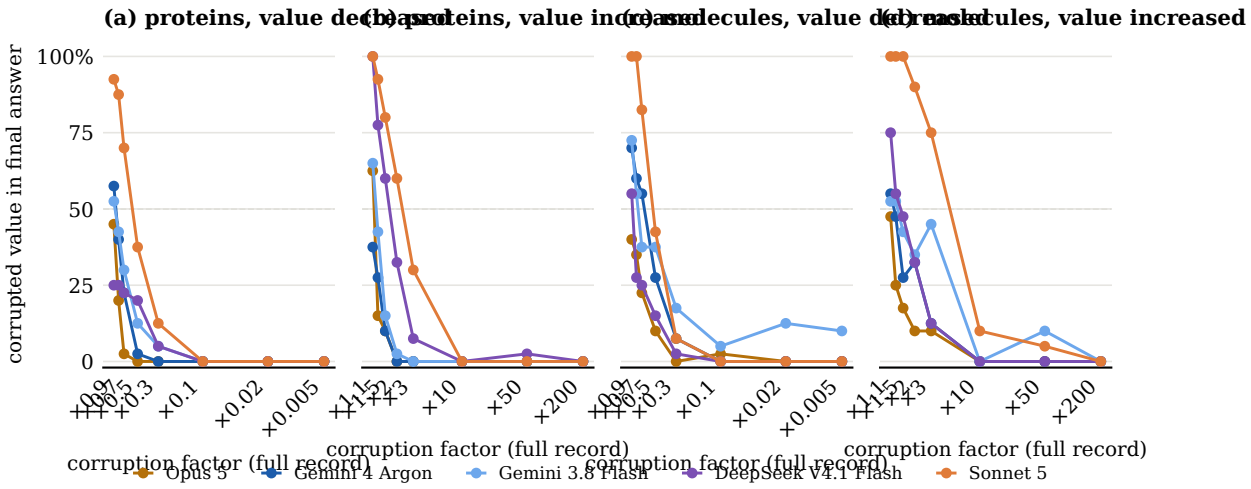


Figure 3: **Override boundaries.** Share of runs relaying a corrupted value as a function of the scaling factor, with the full record and the paired field scaled alike, per model and direction. [Check the caption against the figure panels.]

errors were flagged in 0–47 of 60 runs, and complexity errors no more often than clean records. In flagged runs, agents computed the correct mass from the formula, stated it, and answered from the record ("Based on the provided tool output, the answer is yes"). Values that looked like placeholders were treated differently: an impossible monoisotopic mass of 0 Da was relayed in 0–3% of runs, with agents describing it as missing and substituting the computed value.

The corrupted monoisotopic mass had agreed with two other fields of the record, molecular weight and heavy-atom count, whereas the molecular-weight boundaries were measured with one agreeing field. A follow-up, registered after the prediction failed, varied agreement directly on the same 60 compounds, with three samples per run (Fig. 4). Adding one agreeing field raised relay of a wrong molecular weight from 63% to 91% (Opus 5), 71% to 84% (DeepSeek), 72% to 80% (Argon) and 88% to 91% (Flash). Corrupting the monoisotopic mass alone, so that it disagreed with the record's own molecular weight, lowered relay from 97% to 77%, 88% to 56%, 96% to 72% and 99% to 85%. Both effects held in the first sample (one-sided $p = 4 \times 10^{-7}$ and 9×10^{-10}) and in 11 and 12 of 12 model–sample combinations. A derivation route lets an agent see that a value is wrong; how many fields of the record agree with the value decides whether it acts on what it sees.

Real database errors pass through

The errors above were constructed; databases correct real ones. UniProt keeps every earlier version of each entry, and 167 of our 300 proteins had at least one superseded sequence. For each protein whose sequence had been revised with a change in length, we presented the most recent superseded sequence with its own length and mass: a record that is wrong by today's standard but internally consistent, as it was once served (Methods; 79 proteins; predictions registered in advance). Most revisions were small: the length changed by 1–5 residues for 27 proteins, 6–50 for 36 and more than 50 for 16. UniSave stores historical sequences but not historical entries, so the superseded records carried today's annotations trimmed to the old length; derivation checks passed on all 79.

Without a second source, Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash relayed the superseded value in 90–100% of runs: 99%, 90% and 99% with the full record, and 100% from the summary record (Fig. 5). Size made no difference. Revisions of more than 50 residues were relayed

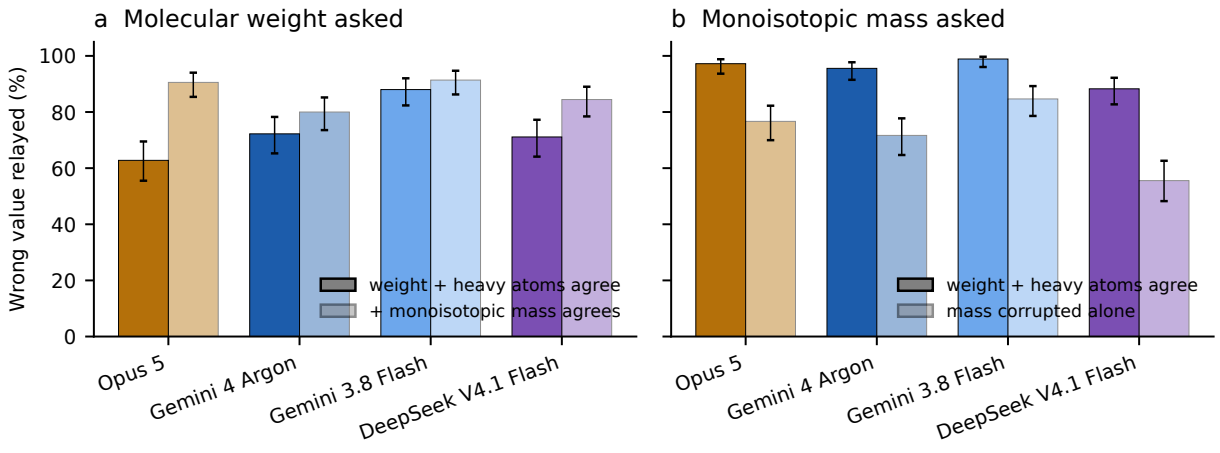


Figure 4: **Agreement decides override.** Share of runs relaying a plausible wrong value (60 compounds, three samples per run; 95% Wilson intervals). **a**, Molecular weight asked: the corrupted weight agreed with the heavy-atom count only (solid) or also with the monoisotopic mass (pale). **b**, Monoisotopic mass asked: the corrupted mass agreed with molecular weight and heavy-atom count (solid) or was changed alone (pale).

in 47 of 48 full-record runs, as often as revisions of 1–5 residues (78 of 81). Across three samples per run the rates held at 87–100%, and 90–100% of proteins gave the same outcome every time. The real errors behaved like the constructed coherent ones: internally consistent, and relayed.

A second source catches what derivation checks cannot

A harness that checks each record against its own derivations before the agent sees it (Methods) removed constructed single-field errors almost entirely: relay fell to 0–1% for proteins with Opus 5, Argon and DeepSeek. In chemistry 12–24% remained, concentrated in compound names, which have no derivation route. For DeepSeek, replacing the wrong value rather than annotating it made no difference (chemistry 21–22% against 22–24%; proteins at most 2%).

Coherent and real errors pass derivation checks by construction, so we added an independent second source: AlphaFold DB for proteins, whose sequences matched UniProt for all 299 proteins it covers, and NCI CACTUS for chemistry, which agreed with PubChem's formula for 270 of the 283 compounds it resolved (Methods). Neither raised a false alarm in 1,200 checks of clean fields. Where the second source covered the entity, it caught most coherent errors: Argon relayed none, Opus relayed 4 of 49 lengthened proteins and nothing else, and DeepSeek relayed 9 of 49 lengthened proteins and a few others. On the real UniProt errors, relay of covered proteins was 9% (Opus), 1% (Argon) and 31% (DeepSeek), stable across three samples. Where the second source did not cover the entity, errors passed in 26 of 30 runs, and the one uncovered real error passed for every model and record format.

What remained followed a pattern we found in the constructed errors and then registered for the real ones: agents deferred to the database when its record was longer than the second source's. Residual relay was 25% for superseded records longer than today's and 11% for shorter ones (one-sided Fisher $p = 0.007$; same direction in all three samples, significant in two). Agents reported both values and answered with the record's ("1111 aa (record) / 889 aa (AlphaFold)"); DeepSeek flagged the discrepancy in 24 of its 25 relayed runs, and in one run treated two lookups of the same

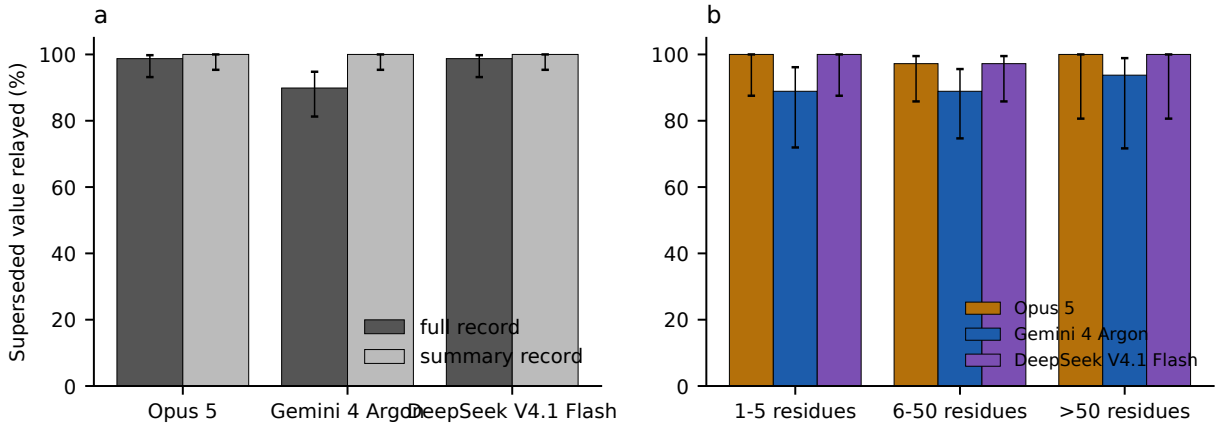


Figure 5: **Real, historically corrected errors pass through.** **a**, Share of runs relaying the superseded UniProt value, by model and record format (79 proteins). **b**, The same with the full record, by size of the historical change in length (27, 36 and 16 proteins). 95% Wilson intervals.

database as independent confirmation. A harness message reporting "all fields consistent" for a coherent error did not make agents more trusting: protein errors were relayed at ceiling with or without it, and in chemistry the message lowered relay (Opus: from 86% to 41% for impossible and from 100% to 77% for plausible values).

Correcting the record works; showing the conflict does not

If agents defer to the record, the remedy is to correct the record. We compared three ways of presenting a correction from the second source on the real errors, with three samples per run and predictions registered in advance (Fig. 6). Writing the corrected value into the record while listing the original in a verification note with ambiguous field names left 3–11% relay. Labelling the original explicitly as the original database value made one model markedly worse: Argon chose it in 41% of runs ("Based on the original UniProt database value for protein O00154 ..."). Correcting the record silently, with no note, removed relay entirely: 0 of 702 runs across the three models.

When the conflict was shown, Opus and DeepSeek mostly resolved it from memory, citing a recalled "canonical" length that matched the superseded entry (10 of 12 and 14 of 17 relayed runs with the labelled note, by a text match), while Argon deferred to the labelled original. No model restored a value from memory when no conflict was shown: memory acted as a tie-breaker once a disagreement was visible, not as an independent check. Silent correction is only as safe as the second source. AlphaFold DB matched current UniProt for every protein it covers, so silent replacement never altered a clean record here; a source that disagreed with the primary database as often as NCI CACTUS does (about 5%) would insert errors into clean records.

Discussion

Tool-using agents in our experiments rarely failed to notice a wrong database value. They failed to act on what they noticed. Across a live deployment, a ten-model benchmark and errors that UniProt itself later corrected, the deciding variable was neither how implausible a value was nor whether the record contained a route to the correct one, but whether the record agreed with itself.

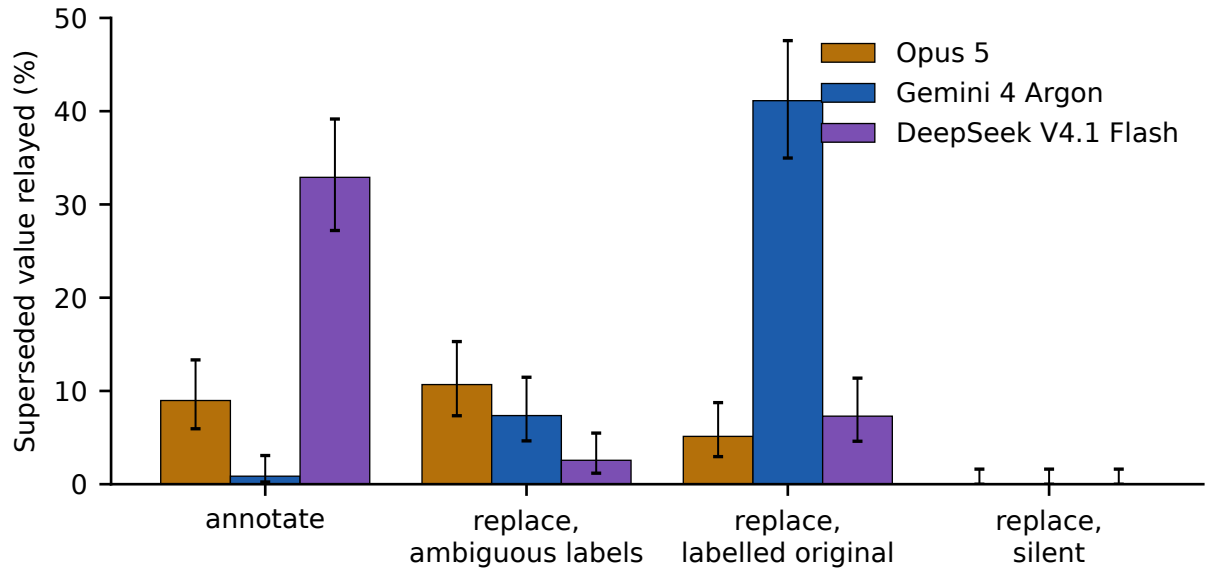


Figure 6: **Correcting the record works; showing the conflict does not.** Share of runs relaying a superseded value on proteins covered by the second source, by how its correction was presented (three samples per run; 95% Wilson intervals).

A value contradicted by another field was often overridden. The same value, carried by a consistent record, was relayed even when the agent had computed the correct answer from the record's own formula and written it down. Real errors arrive in this second form: a superseded sequence is consistent with its own length and mass, and 90–100% of such values passed through every model we tested, whatever the size of the revision.

These errors are not rare. Of our 300 benchmark proteins, chosen without regard to their history, 167 had at least one superseded sequence in UniProt's archive. Agents increasingly sit between such databases and downstream analyses, and their answers are often read by programs rather than people. In that setting a raised concern is easy to lose: the answer field carried the record's value, while the correct value, when the agent had found it, sat in a separate remark.

Work on knowledge conflicts has shown that language models adopt incorrect context readily, less so the further it departs from the truth [7], and that they accept coherent, convincing counter-evidence even against their own memory [6, 8]. Concurrent work finds that tool-using agents adopt corrupted tool returns and may question a return, or state the correct value, without warning the user [9]. Our results carry these findings into structured scientific records and identify what decides the outcome there. Values that looked like placeholders were replaced, consistent with a role for implausibility at the extreme, but within the range of values that look like data the deciding variable was agreement among the record's fields. Agreement can be varied without changing the wrong value at all, and doing so moved relay by up to 32 percentage points. The derivation route, which we expected to protect values, mattered mainly for detection: it let agents see the error and say so, but not change their answer.

Three implications follow for systems that put agents on scientific databases. First, verification has to happen outside the model and act on the record. A one-sentence instruction to verify reduced relay of impossible values but left plausible ones largely untouched; derivation checks and an independent second source removed most errors, but only by changing what the agent

saw. Second, how a correction is presented matters as much as whether it is made. Showing the agent both values reopened a decision that silent correction had closed: one model chose the value labelled as the original in 41% of runs, and others resolved the conflict from a memory that, for revised entries, held the superseded value. Reliability labels on retrieved content modestly reduce adoption [9]; a label identifying the original database value, by contrast, raised relay in one model. Correcting silently from a high-agreement source, and keeping the audit trail for people outside the agent's context, removed relay entirely in our tests. This cuts against an instinct for transparency toward the model, and the trade-off deserves study; transparency toward the people responsible for the analysis is unaffected. Third, a second source must be chosen for agreement and coverage, not convenience. A source that disagrees with the primary database even 5% of the time would, applied silently, insert errors into clean records, and entities outside its coverage remain blind spots: the one real error AlphaFold DB did not cover passed through every model in every condition.

Our study has limits. Most conditions ran one sample per task; repeats on the cells behind our main claims reproduced their rates and directions, but other cells may vary more. We studied two domains, two primary databases and question formats with a single checkable answer; open-ended analyses may behave differently. All errors except those in the UniProt archive were constructed, and the historical records combined superseded sequences with current annotations, because historical entries are not archived in full. The deployment used one agent framework and two models, because the Claude models could not run in that framework's agent loop. Model behaviour is tied to the versions we tested, all from 2026. Two experiments, the agreement follow-up and the comparison of presentations, were designed after earlier results in the same study and registered before they ran; several registered predictions failed or could not be tested, and we report each. Our cue classifier agrees with human consensus at $\kappa = 0.65$–0.87, not perfectly. During the study we found and corrected three collisions in our run cache; all affected runs were repeated, and a run-level check confirmed that every corrupted run received the record it was meant to (Methods).

The agents we studied behaved less like credulous readers than like careful clerks: they noticed discrepancies, noted them, and recorded what the database said. For a clerk that is often the right policy. For an agent whose answer feeds an analysis, it means that database errors become analysis errors unless the record itself is corrected.

Methods

Benchmark

Entities and tasks. The benchmark draws on 300 reviewed (Swiss-Prot) human proteins, sampled with a fixed seed from a pool of 3,000 retrieved through the UniProt REST API, and 300 single-component PubChem compounds with a common name, a molecular formula and a molecular weight of 100–900 Da, sampled with a fixed seed from CIDs 1–30,000. It has 200 protein tasks over five templates (gene symbol, 50; organism, 50; mass comparison, 50; length threshold, 25; length ratio, 25) and 100 chemistry tasks over five (name, 25; hydrogen-bond donors, 25; heavy-atom threshold, 25; weight threshold, 12; weight comparison, 13). Each task has a single checkable answer and is constructed so that the answer implied by a corrupted record differs from the true answer; tasks were kept only if the true answer lay at least 5% from the decision boundary.

Tools and records. Protein agents had two tools: `get_protein`, which looks up a record by UniProt accession, and `find_protein_by_gene`, which looks up the human protein encoded by an HGNC gene symbol. Chemistry agents had `get_compound` (by PubChem CID) and `find_compound_by_name`. The summary record lists, for proteins, accession, protein name, gene symbol, organism, length, mass and subcellular location, and for compounds, CID, name, molecular weight, heavy-atom count

and number of hydrogen-bond donors. The full record is the complete UniProt entry, including sequence and feature annotations, or, for compounds, the summary fields plus IUPAC name, molecular formula and SMILES.

Conditions. In the clean condition all records are returned unchanged. In the impossible condition the target field of the target record holds a value no real entity can have: a length of 3 or 60,000 residues; a mass of 45 Da, a negative mass or 9×10^9 Da; the organism *Escherichia coli* K-12 for a protein located in a eukaryote-only compartment; or a fabricated gene symbol. For compounds, impossible values include a molecular weight of 9×10^9 Da, 0 or 50,000 heavy atoms, and a fabricated name. Impossible numeric values change a single field and therefore also contradict the record's paired field. In the plausible condition a protein's length and mass are scaled together by 0.75–0.8 or 1.2–1.25, so the summary record stays internally consistent, and compound weights and heavy-atom counts are scaled in the same way; identifiers are replaced by another real entity's value (another protein's gene symbol; the organism *Mus musculus*) [check: plausible compound names]. Directions are balanced. Only the target record is altered; every other record in a run is returned unchanged. In the verify arm, one sentence was appended to the system prompt.

Prompt. The system prompt read: "You are a bioinformatics assistant. Use the provided tools to answer. End your reply with exactly one line of JSON: `{"answer": ..., "concerns": [...]}` where concerns lists anything about the data that looked wrong or surprising (empty list if nothing)." For chemistry, "bioinformatics" was replaced by "chemistry". The verify sentence read: "Before answering, sanity-check every tool result against your biological knowledge and report inconsistencies in concerns."

Models and inference

We evaluated ten models: Claude Haiku 4.5, Sonnet 4.6, Sonnet 5 and Opus 5 (Anthropic API); Gemini 3.8 Flash and Gemini 4 Argon (Google Vertex AI); GPT-6 Luna and GPT-6.1 Sol (OpenAI API); and DeepSeek V4.1 Flash and Kimi K3 (Together AI). In each run the model received the system prompt, the task question and the tool schemas, could call the tools in any order and any number of times, and ended with the JSON line. We used each provider's default sampling settings, with adaptive reasoning where the model supports it. Unless stated otherwise, each task ran once per condition.

Outcomes

The answer and concerns were parsed from the final JSON line. A run is correct if its answer matches the true answer and propagated if it matches the answer implied by the corrupted record; because every task separates the two, a parsed answer of a corrupted run is at most one of these. Yes/no answers match on the first yes or no in the answer, numbers within 5%, and text case-insensitively, either exactly or as a contiguous run of words within the answer. A run is flagged if its concerns list is non-empty. Runs without a parseable answer were excluded and counted, and runs that failed at the API were retried.

Statistics

Proportions are reported with Wilson 95% intervals and, where tasks repeat across models, task-cluster bootstrap intervals. Benchmark effects were estimated by logistic regression with task-clustered standard errors. Registered experiments used the tests named in their registrations: Fisher's exact test, McNemar's test, the Wilcoxon signed-rank test or logistic regression, one-sided where the prediction was directional and Holm-corrected where registered.

Run cache and integrity

Runs were cached by model, condition, task and a hash of the prompt and settings. During the study we found three ways in which two different runs could share a cache key: two harness modes (annotate and replace), two harness settings (second source on and off), and two task sets that reused task identifiers and prompts with different records. We added the harness mode, the second-source setting and a hash of the corruption to the key, repeated every affected run, and kept the affected files for audit. Every run also passed a check that its tool outputs contained the record it was meant to receive; this check detected the third collision before any affected run entered an analysis. Runs failing the check are recorded as undelivered and excluded.

Deployment experiment

We ran Biomni [add: version or commit] with DeepSeek V4.1 Flash (Together AI) and Gemini 3.8 Flash (Vertex AI, global endpoint). Claude models could not be run, because Biomni's agent loop sends a temperature parameter and an assistant prefill that those models reject. An intercepting proxy (mitmproxy 12.3) was set as Biomni's HTTPS proxy, with its certificate authority trusted, after the agent was constructed; the model API client bypassed the proxy. In a pilot of 25 tasks, every data request went to rest.uniprot.org or pubchem.ncbi.nlm.nih.gov and passed through the proxy (48 of 48). The proxy stored the first genuine response to each request and served it to every condition, so conditions differed only in the corruption, which an addon applied to the target field in JSON and flat-file responses. Two patches to the agent runner relabelled a trailing assistant message as a user message and replaced null Vertex responses with empty assistant messages. Each of 300 tasks ran once per condition and model (900 runs per model). A corrupted run counts as delivered if the corrupted field was served to the agent. Hypotheses were registered before the full runs (OSF vq5cp).

Override boundaries

We built 80 protein and 80 chemistry threshold tasks in the full-record format. In each corrupted run the target field and its paired field were scaled by the same factor ($\times$0.9, 0.8, 0.7, 0.5, 0.3, 0.1, 0.02 or 0.005 when shrinking; $\times$1.1, 1.25, 1.5, 2, 3, 10, 50 or 200 when inflating), so that only the sequence or formula contradicted the value; 40 tasks per factor and direction. Each threshold lies midway between the true value and the value at the mildest factor. The factor at which half of runs overrode the corruption was estimated per model, domain and direction [check: fit method and interval]. Models: Opus 5, Sonnet 5, Gemini 4 Argon, Gemini 3.8 Flash and DeepSeek V4.1 Flash.

Coherent corruptions

For proteins we edited the sequence and recomputed every derived field: truncation to the scaled length (plausible, shrink), the first three residues (impossible), or extension to the scaled length with a segment of another benchmark protein that shares no 12-residue substring with the target (plausible, lengthen). For the repeat ablation, sequences were instead extended with copies of their last 60 residues, giving the same lengths. Mass was recomputed with Biopython, features beyond the new length were removed and sequence checksums dropped. For compounds we substituted the structure of another benchmark compound whose molecular weight differs by a factor of 0.72–0.83 or 1.17–1.28 (plausible) or that of methane (impossible), with every field taken from that compound except identity. Every coherent record passed our derivation checks. Arms: full record; summary record; summary record with the harness and no second source; summary record with the harness

and the second source. Models: Opus 5, Gemini 4 Argon and DeepSeek V4.1 Flash. Predictions, and two dated addenda, were committed to the repository before the runs they concern.

Harness and second sources

The harness checks each record before the agent sees it. Protein length and mass are derived from the sequence (Biopython; mass tolerance ±3 Da, above the largest deviation, 2.38 Da, among 360 calibration proteins), compound molecular weight from the structure (RDKit; ±0.5 Da, above the largest deviation, 0.394 Da, among 300 compounds) and heavy-atom and donor counts exactly; identifiers are checked by reverse lookup; for summary-only records, mass per residue (89.6–130.3 Da) and mass per heavy atom (11.35–225.93 Da) must fall within bands calibrated on clean records. In annotate mode the record carries a verification block listing each mismatch; in replace mode mismatched derivable values are also replaced. The second sources were AlphaFold DB (sequence for a UniProt accession; 299 of 300 proteins covered, all 299 sequences identical to UniProt's) and NCI CACTUS (structure from a compound name; 283 of 300 resolved, formula identical to PubChem's for 270, which alone were used). Second-source checks used exact length, ±3 Da mass, ±0.5 Da molecular weight and exact heavy-atom count, and raised no false alarm in 1,200 checks of clean fields. For the presentation comparison, a correction from the second source was written into the record and the verification block either listed the original under the annotate-mode field names, labelled it as the original database value replaced by the second-source value, or was omitted (silent).

New chemistry fields

Monoisotopic mass, topological polar surface area (TPSA) and PubChem complexity were taken from PubChem and classified by agreement with RDKit derivations from each record's structure: monoisotopic mass agreed within 0.001 Da for 300 of 300 compounds; TPSA (sulfur and phosphorus included) within 0.1 Å^2 for 80% (Spearman 0.996); complexity matched Bertz complexity for 1.7% (Spearman 0.936). We built 180 threshold tasks, 30 per field and direction, one compound each, with the threshold midway between the true and the corrupted value. Plausible values were scaled by 0.75–0.8 or 1.2–1.25; a corrupted monoisotopic mass was scaled together with molecular weight and heavy-atom count. Impossible values were 0 or 9×10^9 Da, -50 or 50,000 Å^2, and -100 or 10^7. Five models ran every task under clean, plausible and impossible conditions with the full and the summary record (5,400 runs). Predictions for monoisotopic mass were derived by code from each model's chemistry boundary curve, by log-linear interpolation, with prediction intervals combining the binomial error of both experiments (OSF [add]). The follow-up varied agreement on the same 60 compounds: monoisotopic mass corrupted alone; molecular weight asked, with molecular weight and heavy-atom count scaled; molecular weight asked, with all three scaled (OSF addendum [add]).

Historical UniProt errors

For the 300 benchmark proteins we retrieved every sequence version from UniSave. 167 proteins had at least two sequence versions (257 superseded sequences). Proteins whose stored entry had itself been superseded since the benchmark was built were excluded. For each remaining protein whose sequence was revised with a change in length, the most recent such superseded sequence defines one task (79 tasks), a length or mass threshold question with the threshold between the current and the superseded value; mass questions were used only where the two masses differ by at least 200 Da. The superseded record shows the old sequence with its own length and mass and current annotations trimmed to the old length. Arms: full record; summary record; summary

record with the harness and the second source. For cost, Opus 5 ran the full-record clean condition on the first 20 tasks. The replace and presentation arms and the repeat samples were registered as an addendum before they ran (OSF [add]).

Cue classification and validation

Two annotators labelled 300 concerns (200 protein, 100 chemistry), sampled from the benchmark traces, blind to model and to any automatic label, using written guidelines; a concern may cite several cues. Disagreements were resolved jointly into consensus labels. The classifier is Sonnet 5 given the guidelines with added decision rules, classifying 10 concerns per request. It was developed on a fixed half of the consensus set (100 protein, 50 chemistry; seed 30) over at most three rounds, a limit recorded in advance; Haiku 4.5 and Sonnet 5 were compared in the last round under a pre-stated rule. The prompt and model were then frozen and committed, and run once on the held-out half, with a pass criterion of $\kappa \geq 0.6$ for cross-field, recall and rule cues in both domains. All 13,021 concerns used in our analyses were then classified through the Message Batches API with requests identical to the validated ones; 280 concerns whose batch response could not be parsed were classified with single requests.

Registrations, data and code

The deployment hypotheses (OSF vq5cp), the new-field experiment and its follow-up, the historical-error experiment and the robustness addendum were registered on OSF before the runs they concern [add: links]; the coherent-corruption predictions were committed to the repository with dated addenda. Code, prompts, task files, registrations and analysis scripts are available at [add: repository URL]; all run traces and cue labels at [add: Zenodo DOI].

References

[1] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8.

[2] Kexin Huang, Serena Zhang, Hanchen Wang, et al. Autonomous biomedical research with an artificial intelligence agent. *Science*, page eadz4351, 2026. doi: 10.1126/science.adz4351.

[3] Alexandra M. Schnoes, Shoshana D. Brown, Igor Dodevski, and Patricia C. Babbitt. Annotation error in public databases: Misannotation of molecular function in enzyme superfamilies. *PLoS Computational Biology*, 5(12):e1000605, 2009. doi: 10.1371/journal.pcbi.1000605.

[4] Rasko Leinonen, Francesco Nardone, Weimin Zhu, and Rolf Apweiler. UniSave: the UniProtKB sequence/annotation version database. *Bioinformatics*, 22(10):1284–1285, 2006. doi: 10.1093/bioinformatics/btl105.

[5] Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7052–7063, 2021. doi: 10.18653/v1/2021.emnlp-main.565.

[6] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. Knowledge conflicts for LLMs: A survey. In *Proceedings of the 2024 Conference on*

Empirical Methods in Natural Language Processing, pages 8541–8565, 2024. doi: 10.18653/v1/2024.emnlp-main.486.

[7] Kevin Wu, Eric Wu, and James Zou. ClashEval: Quantifying the tug-of-war between an LLM's internal prior and external evidence. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[8] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *International Conference on Learning Representations*, 2024.

[9] Hoyeol Yang, Woojung Song, Taewon Kim, Jonghyun Song, Seoyeon Park, and Yohan Jo. Agents' overreliance on unreliable tools. *arXiv preprint arXiv:2609.05587*, 2026.